

Project Final Report

Our project focused on building an application to examine the Brazilian Olist E-Commerce dataset. This dataset contains millions of records across many CSV files, including customer demographics, order progress, timestamps, product details, item-level transactions, seller information, review data, and payment information. Each table essentially has a different part of the e-commerce pipeline, and all together they form a dataset that models the holistic customer journey. The goal of this project was to implement a mini-data system from scratch, without using pandas or other external data-processing libraries. This first involved designing a CSV parser that handles quoted fields, multi-line records, and empty values. On top of this, we added additional functions like joins, group by aggregations, sorting functions, and category-mapping helpers to combine many Olist tables. Then, we built a Streamlit analytics dashboard that transforms the raw e-commerce data into interactive insights. The dashboard examines real business insights such as states that generate the most orders, product categories with the highest revenue, how payment behavior varies across customers, and many more. The second objective of the project was to address scalability. To accomplish this, we implemented a chunk-based CSV reader that is able to stream large datasets without fully loading them into memory, which is important for how big data frameworks manage resource limitations. Overall, our project demonstrates how we analyzed e-commerce behavior and developed the required underlying infrastructure.

The process begins with the loading pipeline, starting with the CSV parser (`parse_csv`). Our parser had to address real-world irregularities like quoted fields, commas inside quoted text, escaped quotation marks, and records that span multiple physical lines. When a separator is found outside of quotes, it is treated as a true field boundary and inside quotes, it is data. Also,

empty fields are converted to None, which allows our downstream operations to treat missing data. Since some datasets may be too large for memory limits, our loader also has a chunked mode (`parse_csv_in_chunks`). The file is thus streamed in segments of a specific size and each segment is returned as its own DataFrame object. This has the same effect as chunked reading in pandas. The algorithm also has counters and yields blocks when the threshold is reached, resetting internal buffers and continuing the scan. This allows for the assumption that the whole dataset cannot be held in the RAM.

After the raw data is parsed, it is stored internally in a column-oriented format. Such a design allows efficient projection and makes aggregation operations more straightforward. The DataFrame abstraction provides operations such as row filtering, projection, joining across tables, grouping, and aggregation. First, we performed filtering by iterating row indices (`filter_rows` method) and applying a predicate function to each constructed row dictionary. Matching indices are collected, and a new DataFrame is produced by selecting only the corresponding values in each column. Projection is a lot simpler. Since columns are stored independently, projecting onto a subset of columns means creating a new DataFrame that references only the selected column lists.

For grouping, the algorithm constructs a dictionary keyed by tuples representing the group-by values (`groupby` method). Since the DataFrame is scanned sequentially, each row's group key is computed and added to the group bucket. After the groups are formed, the aggregator iterates over and computes count, sum, min, max, and mean statistics. It is important to note that only non-None values are considered during the numeric aggregations.

Next, we implemented the join algorithm. This works by building an index on the right-hand table. This index is a hash map from join key values to lists of row indices where that

key appears. When the index is built, the left-hand table is scanned. For each left row, the algorithm looks up matching rows on the right-hand side. For inner joins, only matches are emitted and for left joins, unmatched rows still contribute output with None placeholders for missing right-hand values. By preparing a result schema that merges columns from both tables and handles name collisions, the algorithm effectively emulates a relational join.

Beyond these operations, we needed to add functionality to map product IDs to Portuguese category names and then to English category names (`add_category_name_to_order_items`) to effectively display the data. To achieve this, a helper pipeline was developed that performs joins between `order_items`, product metadata, and the category translation table. This enrichment procedure demonstrates how relational data distributed across many tables can be combined into a unified view. Overall, we felt that designing these functions (`parse_csv`, `parse_csv_in_chunks`, `filter_rows`, `groupby`, `join`) from scratch gave us a deeper appreciation for the layered abstractions inside modern frameworks.

Our project is built on the Brazilian Olist E-Commerce Public Dataset, a comprehensive collection of roughly one hundred thousand orders made between 2016 and 2018 across multiple Brazilian online marketplaces. Instead of relying on pandas or other high-level libraries, we chose to construct the entire analytics system using our own custom CSV parser and DataFrame engine. This decision required a deep understanding of how raw data is structured, stored, and processed. The dataset includes orders, customers, order items, payments, reviews, products, and a translation table for product categories, all connected through keys such as `customer_id`, `order_id`, and `product_id`. Because these tables are interdependent, our system needed to support reliable joining, grouping, filtering, and aggregation to mimic the functionality of a lightweight data analysis framework.

Using these datasets, we developed an interactive analytics dashboard in Streamlit that allows users to explore different aspects of Olist's business. The Customer Insights view joins orders with customer information to show how many orders come from each state in Brazil, revealing strong demand in regions such as São Paulo and highlighting states where customer activity is much lower. The Product Analytics section enriches order items with product category names by joining the order items table with products and then with the category translation table. This allows us to compute total revenue, quantity sold, and average freight cost for every product category, enabling users to quickly identify the most profitable or highest-volume categories. The Payment Trends tab analyzes how customers choose to pay by grouping payment records according to payment type. The resulting bar chart and pie chart clearly show that credit cards dominate consumer behavior, while the average payment value chart provides further insight into transaction patterns and potential financial partnerships.

The Review Explorer visualizes the distribution of review scores and examines how review score correlates with payment type by creating a heatmap. This makes it possible to detect whether certain payment methods consistently result in poorer customer experiences or higher satisfaction. The Filtered Product Explorer offers a more interactive experience by allowing users to filter results based on customer state, product price range, and the number of product categories to display. Behind the scenes, this view performs multiple joins across orders, customers, order items, products, and category tables, and dynamically filters the data using our custom row-filtering method. Finally, the Time Series view analyzes monthly order volume and revenue by converting timestamp strings into datetime objects, extracting year and month, and aggregating metrics across selected time periods. This provides a clear picture of seasonal trends, growth patterns, and year-by-year performance changes. Together, these integrated views form a

complete end-to-end analytical application that transforms raw CSV files into meaningful and interactive business intelligence.

Designing the DataFrame engine deepened our understanding of how data should be represented internally. Rather than storing rows, we stored data column by column, which made aggregation and filtering more efficient. We implemented projection, row-level filtering, groupby operations, and a variety of aggregation functions including count, sum, mean, min, and max. Building the join operation was a particularly important learning moment because it required constructing a hash map on the right table and expanding each left row according to matching keys. By writing this logic ourselves, we began to see how SQL join algorithms work in practice and why hash joins are so commonly used in database engines. Debugging the join functionality also taught us to think carefully about column naming conflicts, suffix handling, and the behavior of inner versus left joins.

Integrating these backend components into Streamlit pushed us to think more like data product designers. We had to consider how an analyst or business user would want to explore the data and which visualizations would make insights intuitive. Creating multiple coherent views such as Customer Insights, Product Analytics, Payment Trends, Review Explorer, and Time Series helped us understand how to structure an analytics application around clear analytical questions rather than around raw data alone. We also learned how to optimize performance using Streamlit's caching system and how to prevent unnecessary recomputation when working with large datasets.

This project gave us an end-to-end perspective on data systems. We experienced what it means to take raw CSV files all the way through parsing, cleaning, joining, aggregating, and finally turning them into interactive dashboards that support real analytical tasks. It strengthened the skills in data engineering, reinforced the understanding of backend–frontend integration, and improved our ability to communicate insights through visualizations.